

Lecture #2

Instructor: Yunseok Choi
(ys.choi@skku.edu)

NLP Basic

자연어 처리 Natural Language Processing (NLP)

- 인간과 컴퓨터 언어 사이의 상호작용을 처리하는 기술
- 자연어: 인간의 언어(연설, 대화, 문장, 수화 ...)
 - 소셜미디어 포스팅의 텍스트
 - 웹 페이지
 - 병원 처방전
 - 음성 메일의 오디오 데이터
 - 제어 시스템의 명령어
 - ...
- 자연어 처리 응용 분야
 - 기계 번역, 인공지능 스피커, 챗봇, STT (Speech-To-Text)

NLP Libraries in Python

- Popular Python NLP Libraries

- NLTK (Natural Language Toolkit)
- Gensim
- TextBlob

- 이번 수업에서는 NLTK를 사용해 볼 것!

- NLTK 설치 : Anaconda 설치에 포함되어 있음
 - Anaconda 환경이 아닌 경우 "sudo pip install -U nltk" 실행하여 설치

말뭉치 Corpus

- 잘 정리된 텍스트 데이터셋을 “말뭉치” (Corpus)라 부름
 - NLTK는 크고 잘 정리된 텍스트 데이터를 50여개 포함
- 말뭉치 예시
 - 웹 텍스트 코퍼스, 트위터 코퍼스, 셰익스피어 코퍼스, 감성의 양 극(긍정 vs. 부정), Names 코퍼스, 워드넷, 로이터 벤치마크 코퍼스
- 말뭉치 활용처
 - 단어의 출현 횟수를 계산하는 사전
 - 모델 학습과 검증을 위한 데이터

말뭉치 설치

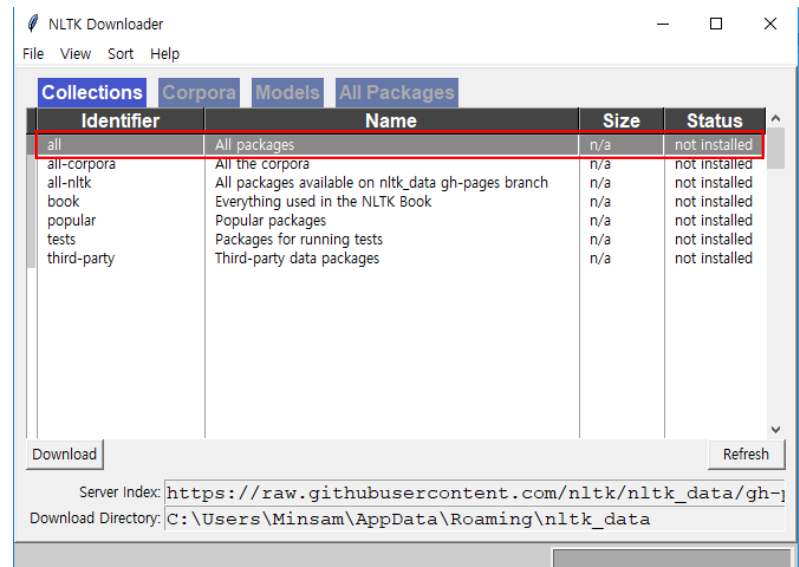
■ NLTK에서 코퍼스 전체 목록 확인하기

- http://www.nltk.org/nltk_data/

■ 말뭉치를 설치해 보자!

- Python에서 명령어 입력 및 실행
- “all”을 더블클릭해서 전체 설치!

```
In [*]: import nltk  
         nltk.download()
```



“names” 말뭉치 사용해 보기

■ 사용할 말뭉치 Import

```
In [3]: from nltk.corpus import names
```

■ 말뭉치가 가지고 있는 단어들 중 10개 출력해 보기

```
In [6]: print(names.words()[:10])
```

```
['Abagael', 'Abigail', 'Abbe', 'Abbey', 'Abbi', 'Abbie', 'Abby', 'Abigael', 'Abigail', 'Abigale']
```

■ 총 name 수 출력

```
In [8]: print( len(names.words()) )
```

```
7944
```

※ 다른 말뭉치도 탐색해 보세요.

NLTK의 텍스트 분석 기능

- 토큰화 (Tokenization)
- 품사 태깅 (POS tagging)
- NER (Name Entity Recognition)
- 어간추출 (Stemming), 표제어 원형 복원 (Lemmatization)

토큰화 Tokenization

- 텍스트 시퀀스가 주어졌을 때, 이를 공백문자로 구분해 **조각으로 나누는 작업**
 - 구두점 (Punctuation), 숫자 (Digit), 이모티콘 (Emoticon) 등 특정 문자들은 삭제
- 나뉜 조각(들)을 **토큰(Token)**이라고 부르며 이후 분석 과정의 기본 단위가 됨
- 토큰을 구성하는 단어(조각) 수에 따른 구분
 - 유니그램 (Unigram) : 토큰이 단어 하나로 구성
 - 바이그램 (Bigram) : 토큰이 2개 연속된 단어로 구성
 - 트라이그램 (Trigram) : 토큰이 3개 연속된 단어로 구성
 - n-그램 (n-gram) : 토큰이 n개 연속된 단어로 구성

유니그램과 바이그램

Input:

Machine learning is awesome, right?

Unigram:

Machine

learning

is

awesome

right

Bigram:

Machine learning

learning is

is awesome

awesome right

품사 태깅 POS tagging

- 주어진 텍스트 시퀀스에 있는 각 단어에 적절한 **품사 태그**를 붙이는 작업
- NLTK 내 “pos_tag(input_token)” 함수 사용

품사	예제
명사 (Noun)	David, machine
대명사 (Pronoun)	them, her
형용사 (Adjective)	awesome, amazing
동사 (Verb)	read, write
부사 (Adverb)	very, quite
전치사 (Preposition)	out, at
접속사 (Conjunction)	and, but
감탄사 (Interjection)	unfortunately, luckily
관사 (Article)	a, the

NER Name Entity Recognition

- 텍스트 시퀀스가 주어졌을 때, 단어 또는 구문을 **사람 이름, 회사 이름, 지역 이름**처럼 명확한 카테고리에 지정하고 식별하는 작업
- 예)
 - 성균관대학교 → 대학교
 - 최윤석 → 사람
 - 경기도 → 지역

어간 추출 Stemming 표제어 원형 복원 lemmatization

- 어간 추출 어근에서 변화되거나 파생된 단어를 원형으로 되돌리는 작업
 - 예) “machines” 의 어간 → “machin”
“learning”, “learned”의 어간 → “learn”
- 표제어 원형 복원: 어간 추출보다 좁은 의미의 작업
 - 예) 명사의 원형만 복원
추출 결과 원형이 불완전하면 사전에서 가장 유사한 명사를 선택

어간 추출기 사용하기

```
In [10]: from nltk.stem.porter import PorterStemmer  
porter_stemmer = PorterStemmer()
```

```
In [11]: porter_stemmer.stem('machines')
```

```
Out[11]: 'machin'
```

```
In [12]: porter_stemmer.stem('learning')
```

```
Out[12]: 'learn'
```

표제어 원형 복원기 사용하기

```
In [13]: from nltk.stem import WordNetLemmatizer  
         lemmatizer = WordNetLemmatizer()
```

```
In [14]: lemmatizer.lemmatize('machines')
```

```
Out[14]: 'machine'
```

```
In [15]: lemmatizer.lemmatize('learning')
```

```
Out[15]: 'learning'
```

Text Data "newsgroup20"

"newsgroups" 데이터

- 사이킷런에 있는 20 newsgroup 데이터를 분석해 보자!
 - 20개 온라인 뉴스그룹을 대상으로 20,000개의 포스트 데이터 존재
 - 뉴스그룹은 특정 토픽에 대해 질의 응답할 수 있는 인터넷 공간 (인터넷 포럼)
 - 학습 데이터와 테스트 데이터로 나누어져 있으며,
두 데이터는 특정 날짜를 기준으로 나뉘짐

- | | | |
|----------------------------|----------------------|--------------------------|
| • comp.graphics | • rec.sport.baseball | • talk.politics.misc |
| • comp.os.ms-windows.misc | • rec.sport.hockey | • talk.politics.guns |
| • comp.sys.ibm.pc.hardware | • sci.crypt | • talk.politics.mideast |
| • comp.sys.mac.hardware | • sci.electronics | • talk.religion.misc |
| • comp.windows.x | • sci.med | • alt.atheism |
| • rec.autos | • sci.space | • soc.religion.christian |
| • rec.motorcycles | • misc.forsale | |

데이터 확보

- 사이킷런에서 제공하는 함수를 통해 데이터셋 확보
 - 코드를 처음 실행해서 다운로드가 되면 캐싱(caching)을 통해 이후 실행 시에는 다시 다운로드하는 것이 아니라 캐싱된 데이터를 빠르게 사용할 수 있도록 지원함

```
In [16]: from sklearn.datasets import fetch_20newsgroups
```

```
In [17]: groups = fetch_20newsgroups()
```

Downloading 20news dataset. This may take a few minutes.

Downloading dataset from <https://ndownloader.figshare.com/files/5975967> (14 MB)

데이터 살펴보기

- 앞선 코드에서 데이터 Import에 성공했다면, groups라는 데이터 객체를 통해 포스트 데이터에 접근 가능
 - 이름(groups)은 앞선 코드에서 얼마든지 변경 가능
- groups 객체는 Python Dictionary 객체
 - 키와 값을 지님

```
In [18]: groups.keys()
```

```
Out[18]: dict_keys(['data', 'filenames', 'target_names', 'target', 'DESCR', 'description'])
```

<groups 객체의 key 값들 확인>

데이터 살펴보기 : 뉴스 텍스트

- “data” key에 실제 포스트 텍스트 데이터가 리스트로 들어 있고 index값을 통해 텍스트 하나 하나에 접근할 수 있음

```
In [15]: groups.data[0]
```

```
Out[15]: "From: lerxst@wam.umd.edu (where's my thing)\nSubject: WHAT car is this!?\n\nNntp-Posting-Host: rac3.wam.umd.edu\nOrganization: University of Maryland, College Park\nLines: 15\n\n I was wondering if anyone out there could enlighten me on this car I saw\nthe other day. It was a 2-door sports car, looked to be from the late 60s/\nnearly 70s. It was called a Bricklin. The doors were really small. In addition,\nthe front bumper was separate from the rest of the body. This is \nall I know. If anyone can tell me a model name, engine specs, years\nof production, where this car is made, history, or whatever info you\nhave on this funky looking car, please e-mail.\n\nThanks,\n- IL\n\n ---- brought to you by your neighborhood Lerxst ----\n\n\n"
```

데이터 살펴보기 : 소속 뉴스 그룹

- “target” key에는 각 포스트 텍스트의 뉴스 그룹 번호가 들어가 있음

- [코드] 제일 처음 데이터(Index = 0)는 뉴스 그룹 번호 7번을 가지고 있음
- 그룹 번호는 0~19까지 존재

첫번째 포스트 텍스트의 소속 그룹

- 각 그룹 번호에 대한 뉴스 그룹 이름을 보고 싶으면 “target_names” key를 참조하면 됨

```
In [16]: groups.target[0]
```

```
Out[16]: 7
```

```
In [17]: groups.target_names
```

```
Out[17]: ['alt.atheism',  
          'comp.graphics',  
          'comp.os.ms-windows.misc',  
          'comp.sys.ibm.pc.hardware',  
          'comp.sys.mac.hardware',  
          'comp.windows.x',  
          'misc.forsale',  
          'rec.autos',  
          'rec.motorcycles',  
          'rec.sport.baseball',  
          'rec.sport.hockey',  
          'sci.crypt',  
          'sci.electronics',  
          'sci.med',  
          'sci.space',  
          'soc.religion.christian',  
          'talk.politics.guns',  
          'talk.politics.mideast',  
          'talk.politics.misc',  
          'talk.religion.misc']
```

Text Analysis

특성 생각해 보기: 단어 출현

- 어떤 단어들이 출현 했는지는 해당 포스트 텍스트가 무엇인지를 알려주는 좋은 특성이다.
- 단어 출현도 다양하게 정량화 될 수 있다.
 - 단어의 출현 여부
 - 단어의 출현 빈도
 - 관련 측정값
 - ...

특성 생각해 보기: 단어 출현

- 어떤 단어들이 출현 했는지는 해당 포스트 텍스트가 무엇인지를 알려주는 좋은 특성이다.
- 단어 출현도 다양하게 정량화 될 수 있다.
 - 단어의 출현 여부
 - 단어의 출현 빈도
 - 관련 측정값
 - ...

또한 상황에 따라서 전체 단어가 아니라 특정 단어들이 중요할 수 있다.

- 정보가 거의 없는 단어들은 제외 (예: a, the, are)
→ 이러한 단어를 '불용어' Stopword라 한다.
- 명사, 동사, 형용사, 부사 ...

텍스트 특성: TF (Term Frequency)

- 텍스트(예: 문서)를 표현하는 대표적인 특성으로 단어의 **출현 빈도(Term Frequency)**가 있다.
 - 문서가 존재하는 특성 공간이 각 단어들로 표현된다.

	자동차	기차	커피	쿠키	...
문서 1	3	4	0	0	
문서 2	3	3	0	1	
문서 3	1	1	7	6	
...	...	...			

텍스트 특성: TF-IDF

- 대부분의 문서에서 일반적으로 많이 나오는 단어들은 TF 값이 높아도 문서의 고유한 특징을 잘 표현 하지 않는다.
- 이점을 고려하여 여러 문서에서 나온 단어에 페널티를 주는 방식이 TF-IDF (Term Frequency-Inverse Document Frequency)

$$TF_IDF = TF \cdot \log \frac{n_D}{1 + n_t}$$

n_D : 전체 문서 수

n_t : 단어 t 가 나온 문서 수

텍스트 특성: TF-IDF 예

TF

	자동차	기차	커피	쿠키	...
문서 1	3	4	0	0	
문서 2	3	3	0	1	
문서 3	1	1	7	6	

IDF

자동차	기차	커피	쿠키	...
$\log(3/4)$	$\log(3/4)$	$\log(3/2)$	$\log(3/3)$	

TF-IDF

	자동차	기차	커피	쿠키	...
문서 1	$3 \cdot \log(3/4)$	$4 \cdot \log(3/4)$	$0 \cdot \log(3/2)$	$0 \cdot \log(3/3)$	
문서 2	$3 \cdot \log(3/4)$	$3 \cdot \log(3/4)$	$0 \cdot \log(3/2)$	$1 \cdot \log(3/3)$	
문서 3	$1 \cdot \log(3/4)$	$1 \cdot \log(3/4)$	$7 \cdot \log(3/2)$	$6 \cdot \log(3/3)$	

문서 모델: Bag of Words 모델

- 이렇게 문서를 수학적으로 표현한 것을 **문서 모델**이라 한다.
- 문서에서 단어의 전체 순서를 중요하게 보지 않으면 문서를 일종의 **단어 집합(bag of words)**으로 표현할 수 있다.
 - 예: 문서1 → 모델: {'단어1': 3, '단어2': 4 ...}
- 단순하지만 실제로 상당히 잘 작동 😊
- 단어 순서나 스피치 태그 등을 활용하는 방법도 존재

특성 생각해 보기: 그외에..

- 단어 출현

- 단어의 출현 여부
- 단어의 출현 빈도
- 관련 측정값
- ...

- 단어 등장 순서

- 텍스트 길이 : `len()` 함수 사용

- ...

```
In [18]: len(groups.data[0])
```

```
Out[18]: 721
```

```
In [19]: len(groups.data[1])
```

```
Out[19]: 858
```

Data Visualization

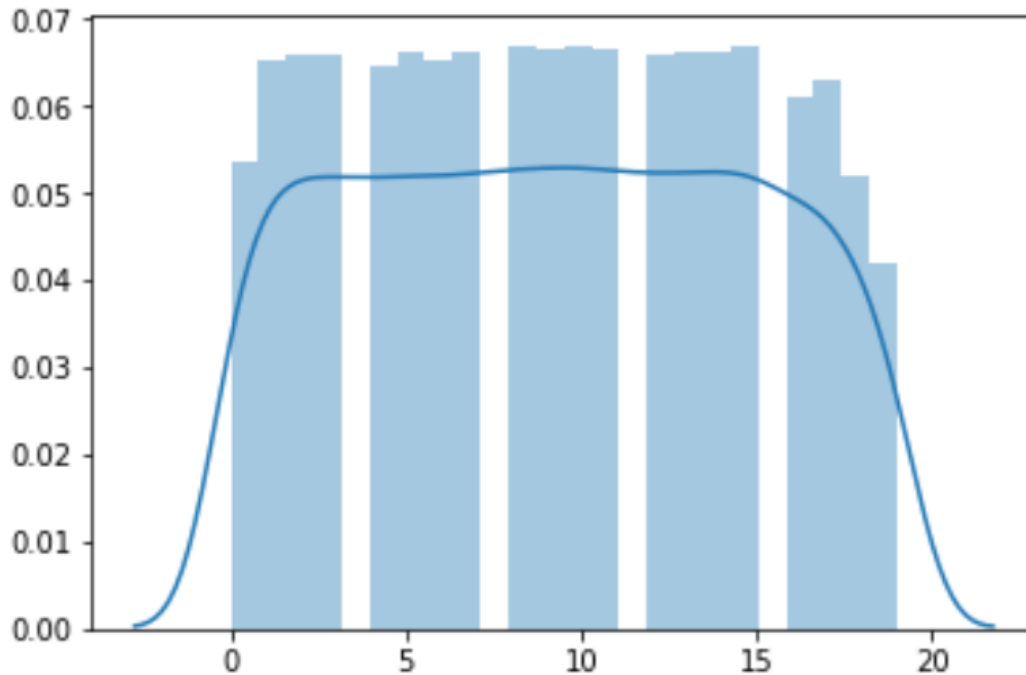
데이터 시각화 Visualization

- 데이터가 준비되었다면 데이터의 특징을 파악하는 것이 필요
 - 데이터가 얼마나 잘 구조화 되어 있는가?
 - 예상되는 이슈는?
 - 비정상적인 데이터가 있는지?
 - 특정 카테고리에 데이터가 몰려 있는지?
 - ...
- 데이터 스케일이 클 수록 **시각화**를 통해 이해하는 것이 유용
- 시각화는 Python의 "**seaborn**", "**matplotlib**" 패키지를 사용
 - Anaconda에 포함
 - 혹은 `pip install --upgrade matplotlib`, `pip install --upgrade seaborn`으로 설치

데이터 분포 확인

```
import seaborn as sns  
sns.distplot(groups.target)
```

<matplotlib.axes._subplots.AxesSubplot at 0x21687f40198>



Newsgroup 데이터는 20개 그룹에 대해 분포가 고르구나!

단어 빈도수 확인

- "CountVectorizer" 클래스를 사용해서
Newsgroup20 데이터의 단어 출현 빈도수(TF)를 시각화 해보자!

<CountVectorizer 클래스 생성자 파라미터>

Constructor parameter	Default	Example values	Description
ngram_range	(1,1)	(1, 2), (2, 2)	Lower and upper bound of the n-grams to be extracted in the input text
stop_words	None	english, [a, the, of], None	Which stop word list to use. If None, do not filter stop words.
lowercase	True	True, False	Whether to use lowercase characters.
max_features	None	None, 500	If not None, consider only a limited number of features.
binary	False	True, False	If True sets non-zero counts to 1.

단어 빈도수 확인

1. 필요한 Python Module들 Import!

```
from sklearn.feature_extraction.text import CountVectorizer
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_20newsgroups
```

단어 빈도수 확인

2. Bag of Words 모델 만들기 (전체 데이터에 대한)

```
cv = CountVectorizer(stop_words="english", max_features=500)

groups = fetch_20newsgroups()

transformed = cv.fit_transform(groups.data)

print(cv.get_feature_names())
```

단어 빈도수 확인

`print(cv.get_feature_names())` 실행 결과

```
['00', '000', '0d', '0t', '10', '100', '11', '12', '13', '14', '145', '15', '16', '17', '18', '19', '1993', '1d9', '20', '21', '22', '23', '24', '25', '26', '27', '28', '29', '30', '31', '32', '33', '34', '34u', '35', '40', '45', '50', '55', '80', '92', '93', '___', '___', 'a86', 'able', 'ac', 'access', 'act', 'ually', 'address', 'ago', 'agree', 'al', 'american', 'andrew', 'answer', 'anybody', 'apple', 'application', 'apr', 'april', 'area', 'argument', 'armenian', 'armenians', 'article', 'ask', 'asked', 'att', 'au', 'available', 'away', 'ax', 'b8f', 'bad', 'based', 'believe', 'berkeley', 'best', 'better', 'bible', 'big', 'bike', 'bit', 'black', 'board', 'body', 'book', 'box', 'buy', 'ca', 'california', 'called', 'came', 'canada', 'car', 'card', 'care', 'case', 'cause', 'cc', 'center', 'certain', 'certainly', 'change', 'check', 'children', 'chip', 'christ', 'christian', 'christians', 'church', 'city', 'claim', 'clinton', 'clipper', 'cmu', 'code', 'college', 'color', 'colorado', 'columbia', 'com', 'come', 'comes', 'company', 'computer', 'consider', 'contact', 'control', 'copy', 'correct', 'cost', 'country', 'couple', 'course', 'cs', 'current', 'cwru', 'data', 'dave', 'david', 'day', 'days', 'db', 'deal', 'death', 'department', 'dept', 'did', 'didn', 'difference', 'different', 'disk', 'display', 'distribution', 'division', 'dod', 'does', 'doesn', 'doing', 'don', 'dos', 'drive', 'driver', 'drivers', 'earth', 'edu', 'email', 'encryption', 'end', 'engineering', 'especially', 'evidence', 'exactly', 'example', 'experience', 'fact', 'fait', 'h', 'faq', 'far', 'fast', 'fax', 'feel', 'file', 'files', 'following', 'free', 'ftp', 'g9v', 'game', 'games', 'general', 'getting', 'given', 'gmt', 'god', 'going', 'good', 'got', 'gov', 'government', 'graphics', 'great', 'group', 'groups', 'guess', 'gun', 'guns', 'hand', 'hard', 'hardware', 'having', 'healt', 'h', 'heard', 'hell', 'help', 'hi', 'high', 'history', 'hockey', 'home', 'hope', 'host', 'house', 'hp', 'human', 'ibm', 'idea', 'image', 'important', 'include', 'including', 'info', 'information', 'instead', 'institute', 'interested', 'internet', 'isn', 'israel', 'israeli', 'issue', 'james', 'jesus', 'jewish', 'jews', 'jim', 'john', 'just', 'keith', 'key', 'keys', 'keywords', 'kind', 'know', 'known', 'large', 'later', 'law', 'left', 'let', 'level', 'life', 'like', 'likely', 'line', 'lines', 'list', 'little', 'live', 'll', 'local', 'long', 'look', 'looking', 'lot', 'love', 'low', 'ma', 'mac', 'machine', 'mail', 'major', 'make', 'makes', 'making', 'man', 'mark', 'matter', 'max', 'maybe', 'mean', 'means', 'memory', 'men', 'message', 'michael', 'mike', 'mind', 'mit', 'money', 'mr', 'ms', 'na', 'nasa', 'national', 'need', 'net', 'netcom', 'network', 'new', 'news', 'newsreader', 'nice', 'nntp', 'non', 'note', 'number', 'numbers', 'office', 'oh', 'ohio', 'old', 'open', 'opinions', 'order', 'org', 'organization', 'original', 'output', 'package', 'paul', 'pay', 'pc', 'people', 'period', 'person', 'phone', 'pitt', 'pl', 'place', 'play', 'players', 'point', 'points', 'police', 'possible', 'post', 'posting', 'power', 'president', 'press', 'pretty', 'price', 'private', 'probably', 'problem', 'problems', 'program', 'programs', 'provide', 'pub', 'public', 'question', 'questions', 'quit', 'e', 'read', 'reading', 'real', 'really', 'reason', 'religion', 'remember', 'reply', 'research', 'right', 'rights', 'robert', 'run', 'running', 'said', 'sale', 'san', 'saw', 'say', 'saying', 'says', 'school', 'science', 'screen', 'scsi', 'season', 'second', 'security', 'seen', 'send', 'sense', 'server', 'service', 'services', 'set', 'similar', 'simple', 'simply', 'single', 'size', 'small', 'software', 'sorry', 'sort', 'sound', 'source', 'space', 'speed', 'st', 'standard', 'start', 'started', 'state', 'states', 'steve', 'stop', 'stuff', 'subject', 'summary', 'sun', 'support', 'sure', 'systems', 'talk', 'talking', 'team', 'technology', 'tell', 'test', 'text', 'thanks', 'thing', 'things', 'think', 'thought', 'time', 'times', 'today', 'told', 'took', 'toronto', 'tried', 'true', 'truth', 'try', 'trying', 'turkish', 'type', 'uiuc', 'uk', 'understand', 'university', 'unix', 'unless', 'usa', 'use', 'used', 'user', 'using', 'usually', 'uucp', 've', 'version', 'video', 'view', 'virginia', 'vs', 'want', 'wanted', 'war', 'washington', 'way', 'went', 'white', 'win', 'window', 'windows', 'won', 'word', 'words', 'work', 'working', 'works', 'world', 'wouldn', 'write', 'writes', 'wrong', 'wrote', 'year', 'years', 'yes', 'york']
```

선택된 상위 500개 단어들이 보여진다.

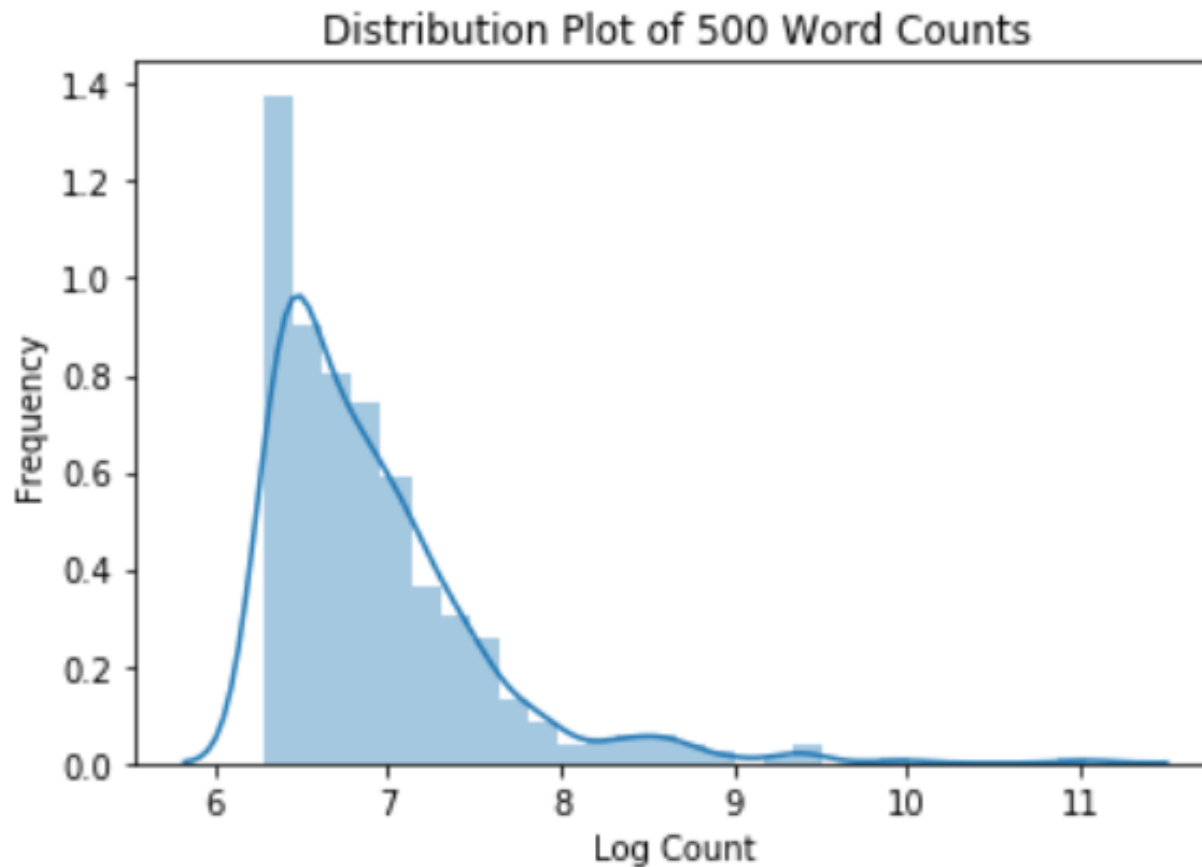
단어 빈도수 확인

3. 단어 빈도수 시각화

```
sns.distplot(np.log(transformed.toarray().sum(axis=0)))  
  
plt.xlabel('Log Count')  
  
plt.ylabel('Frequency')  
  
plt.title('Distribution Plot of 500 Word Counts')  
  
plt.show()
```

단어 빈도수 확인

`plt.show()` 실행 결과



TF-IDF로 표현하기

- “TfidfVectorizer” 클래스를 사용하면
Newsgroup20 데이터를 TF-IDF 특성 공간에 표현할 수 있다.

```
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_20newsgroups

tfidf = TfidfVectorizer(stop_words="english", max_features=500)

groups = fetch_20newsgroups()

transformed = tfidf.fit_transform(groups.data)
```

Text Preprocessing

텍스트 전처리 Preprocessing

- 숫자, 특수문자 처리
 - 앞서 추출된 단어들을 보면 '00', '000'처럼 문자 외 단어 제거
- 어간 추출 (Stemming), 표제어 원형 복원 (Lemmatization)
 - Word와 words와 같이 어근이 같으면 같은 단어로 간주
 - Stemming은 결과 자체가 단어 원형으로 나오지는 않을 수 있다.
 - Lemmatization은 Stemming의 결과에서 가장 가까운 원형을 찾아 반환하기 때문에 정확도는 더 높다. (시간은 오래 걸림)
- 대소문자 처리 → 통일
- 고유명사 (Name Entity) 처리 → 삭제 혹은 포함

텍스트 전처리 Preprocessing

- Lemmatization, 문자만 선택, Name Entity 제거
 - 필요한 Python Module들 Import!

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.datasets import fetch_20newsgroups
from nltk.corpus import names
from nltk.stem import WordNetLemmatizer
```

텍스트 전처리 Preprocessing

- Lemmatization, 문자만 선택, Name Entity 제거

```
cv = CountVectorizer(stop_words="english", max_features=500)

groups = fetch_20newsgroups()

cleaned = []

all_names = set(names.words())

lemmatizer = WordNetLemmatizer()

for post in groups.data:

    lemmatized_list = [ lemmatizer.lemmatize( word.lower() )
                        for word in post.split()
                        if word.isalpha() and word not in all_names ]

    cleaned.append(' '.join(lemmatized_list))
```

단어가 알파벳(즉 문자)인지 체크

텍스트 전처리 Preprocessing

Tokenization 결과

```
transformed = cv.fit_transform(cleaned)
```

```
print(cv.get_feature_names())
```

```
['able', 'accept', 'access', 'according', 'act', 'action', 'actually', 'ad  
swer', 'anybody', 'apple', 'application', 'apr', 'arab', 'area', 'argument  
t', 'available', 'away', 'bad', 'based', 'basic', 'belief', 'believe', 'bes  
'box', 'build', 'bus', 'business', 'buy', 'ca', 'california', 'called', 'ca  
ainly', 'chance', 'change', 'check', 'child', 'chip', 'christian', 'church  
ng', 'command', 'comment', 'common', 'communication', 'company', 'computer  
py', 'correct', 'cost', 'country', 'couple', 'course', 'cover', 'create', '  
esign', 'device', 'did', 'difference', 'different', 'discussion', 'disk', '  
'earth', 'easy', 'effect', 'email', 'encryption', 'end', 'engineering', 'er  
t', 'experience', 'explain', 'face', 'fact', 'faq', 'far', 'fast', 'federal  
e', 'friend', 'ftp', 'function', 'game', 'general', 'getting', 'given', 'gr  
eek', 'ground', 'group', 'guess', 'gun', 'guy', 'ha', 'hand', 'hard', 'harc  
tory', 'hit', 'hockey', 'hold', 'home', 'hope', 'house', 'human', 'ibm', '-
```

Lab 2-1: Text Processing

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

Supervised Learning

Supervised Learning

- A quiz

Math Quiz #1 - Teacher's Answer Key

1) $2 \ 4 \ 5 = 3$

2) $5 \ 2 \ 8 = 2$

3) $2 \ 2 \ 1 = 3$

4) $4 \ 2 \ 2 = 6$

5) $6 \ 2 \ 2 = 10$

6) $3 \ 1 \ 1 = 2$

7) $5 \ 3 \ 4 = 11$

8) $1 \ 8 \ 1 = 7$

- What would be the answer for the following data?

- $6 \ 9 \ 10 = ?$

- 44, how?

Supervised Learning

- Supervised learning: A task of learning a function that maps an input to an output based on example input-output pairs
- Example

Bedrooms	Sq. feet	Neighborhood	Sale price
3	2000	Normaltown	\$250,000
2	800	Hipsterton	\$300,000
2	850	Normaltown	\$150,000
1	550	Normaltown	\$78,000
4	2000	Skid Row	\$150,000

[training data]

Bedrooms	Sq. feet	Neighborhood	Sale price
3	2000	Hipsterton	???

[test data]

Supervised Learning

- Example

Bedrooms	Sq. feet	Neighborhood	Sale price
3	2000	Normaltown	\$250,000
2	800	Hipsterton	\$300,000
2	850	Normaltown	\$150,000
1	550	Normaltown	\$78,000
4	2000	Skid Row	\$150,000

Bedrooms	Sq. feet	Neighborhood	Sale price
3	2000	Hipsterton	???

- An example code by an expert

```
def estimate_house_sales_price(num_of_bedrooms, sqft, neighborhood):
    price = 0
    price_per_sqft = 200
    if neighborhood == "hipsterton":
        price_per_sqft = 400
    elif neighborhood == "skid row":
        price_per_sqft = 100

    price = price_per_sqft * sqft
    if num_of_bedrooms == 0:
        price = price — 20000
    else:
        price = price + (num_of_bedrooms * 1000)
    return price
```

Supervised Learning

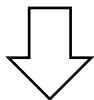
- Example

Bedrooms	Sq. feet	Neighborhood	Sale price
3	2000	Normaltown	\$250,000
2	800	Hipsterton	\$300,000
2	850	Normaltown	\$150,000
1	550	Normaltown	\$78,000
4	2000	Skid Row	\$150,000

Bedrooms	Sq. feet	Neighborhood	Sale price
3	2000	Hipsterton	???

- An example solution by a computer

```
def estimate_house_sales_price(num_of_bedrooms, sqft, neighborhood):  
    price = <컴퓨터, 나 대신 수학적 식 좀 만들어줘>  
    return price
```



```
def estimate_house_sales_price(num_of_bedrooms, sqft, neighborhood):  
    price = 0  
    price += num_of_bedrooms * .841231951398213  
    price += sqft * 1231.1231231  
    price += neighborhood * 2.3242341421  
    price += 201.23432095  
    return price
```

Supervised Learning

- Another example: spam filtering

	viagra	learning	the	dating	nigeria	<i>spam?</i>
$\vec{x}_1 = ($	1	0	1	0	0)	$y_1 = 1$
$\vec{x}_2 = ($	0	1	1	0	0)	$y_2 = -1$
$\vec{x}_3 = ($	0	0	0	0	1)	$y_3 = 1$

- Instance space $x \in X$ ($|X| = n$ data points)
 - Binary or real-valued feature vector x of word occurrences
 - d features (words + other things, $d \sim 100,000$)
- Class $y \in Y$
 - y : Spam (+1), Ham (-1)
- Goal: Estimate a function $f(x)$ so that $y = f(x)$

Supervised Learning

- Would like to do prediction: estimate a function $f(x)$ so that $y = f(x)$
 - Where y can be
 - Real number: Regression
 - Categorical: Classification
 - Complex object: Ranking of items, etc.
 - Data is labeled:
 - Have many pairs $\{(x, y)\}$
 - ✓ x ... vector of binary, categorical, real valued features
 - ✓ y ... class ($\{+1, -1\}$, or a real number)

Unsupervised Learning

- A type of **machine learning** algorithm used to draw inferences from datasets consisting of input data without labeled responses
 - Data is unlabeled

Bedrooms	Sq. feet	Neighborhood
3	2000	Normaltown
2	800	Hipsterton
2	850	Normaltown
1	550	Normaltown
4	2000	Skid Row

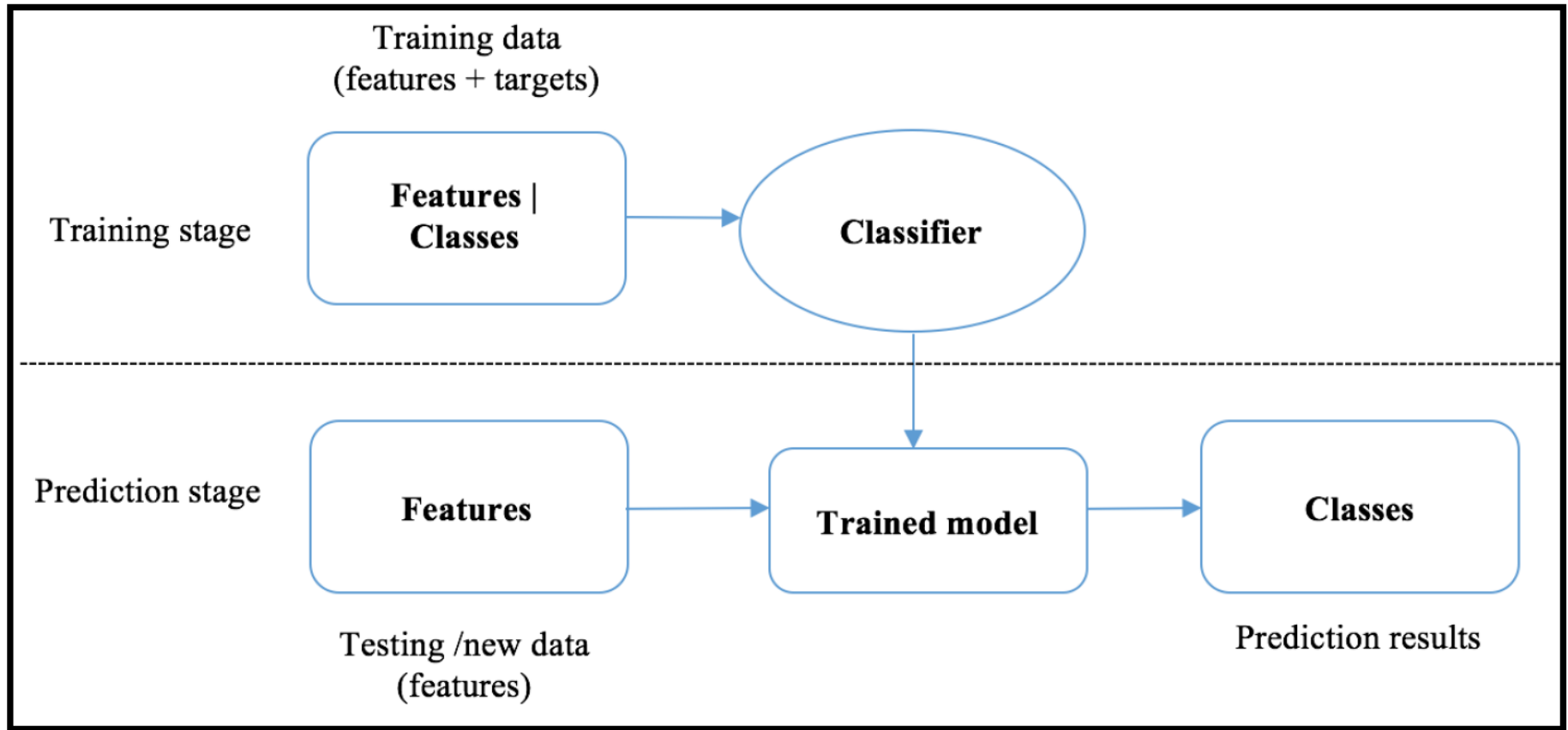
You may want to group houses into several clusters!

Classification

분류 Classification

- 관측값(=특성)이 포함된 학습 데이터와 이 데이터에 연관된 카테고리 결과(=클래스 레이블)가 주어지고,
관측값을 보고 해당 데이터를 적절한 카테고리에
올바르게 매핑시키는 룰을 학습하는 기법
 - 대표적인 지도학습 기법

분류의 두 단계

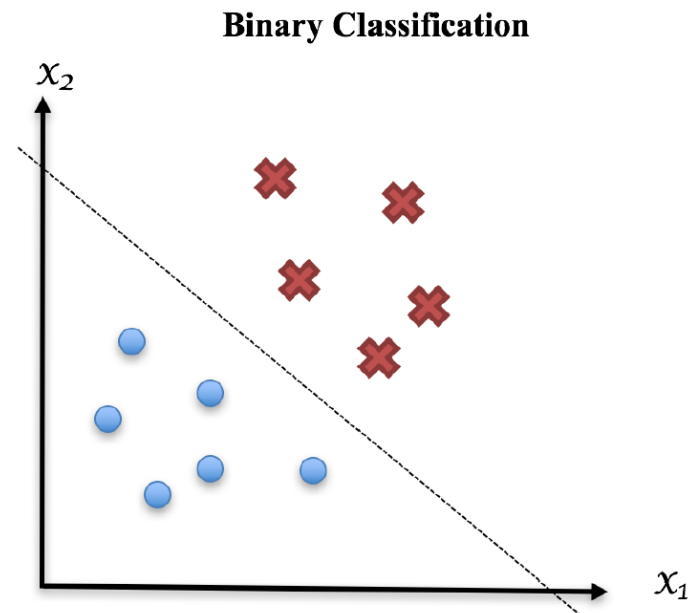


분류의 유형

- 이진 클래스 분류 Binary Classification
 - 관측값을 두 개의 클래스 중 하나에 할당하는 문제
- 다중 클래스 분류 Multiclass Classification
 - 관측값을 세 개 이상의 클래스 중 하나에 할당하는 문제
- 다중 레이블 분류 Multi-label Classification
 - 관측값을 여러 개 클래스에 할당하는 문제

이진 클래스 분류 Binary Classification

- 관측값을 두 개의 클래스 중 하나에 할당하는 문제
 - 스팸 메일 or 정상 메일로 판별하는 '스팸 메일 필터링'
 - CRM 시스템에서 Churn 예측 (이탈자 예측)
 - 마케팅 광고 분야에서 온라인 광고 클릭 스루 예측 (Click-through)
 - 조기 암 진단 기법

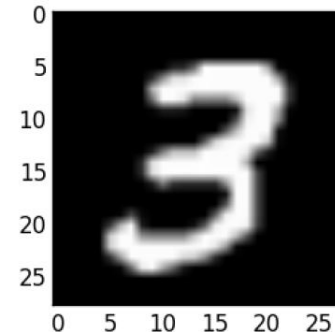
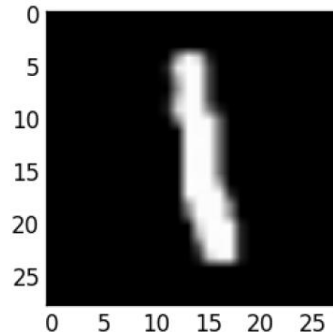
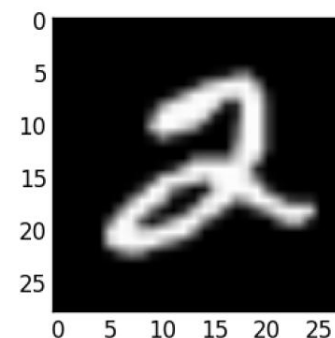
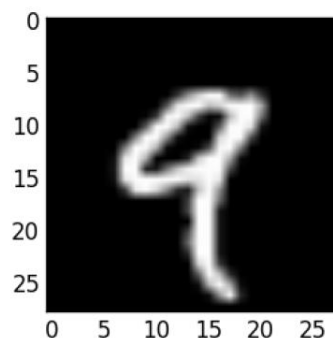
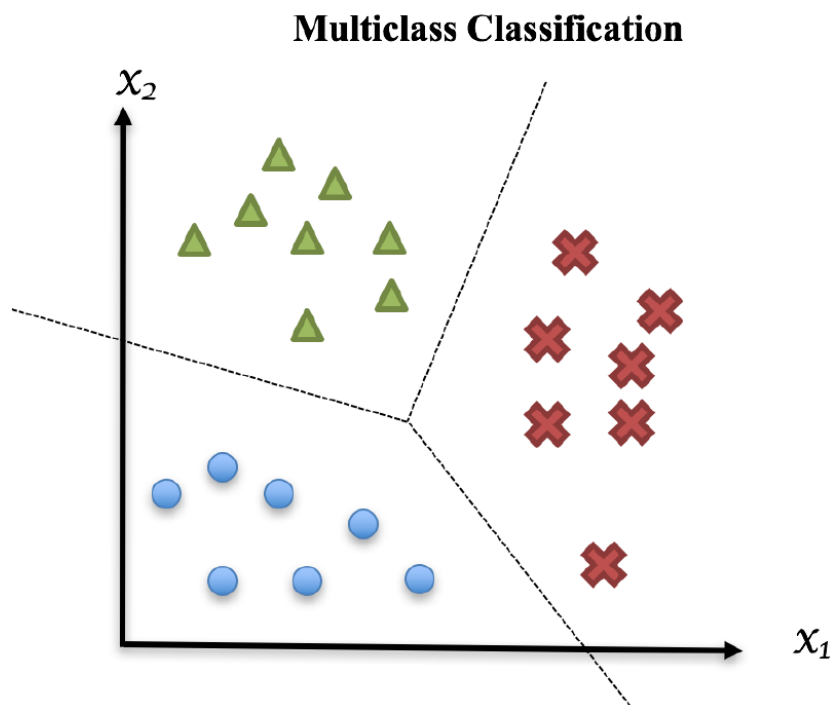


다중 클래스 분류 Multiclass Classification

- 관측값을 세 개 이상의 클래스 중 하나에 할당

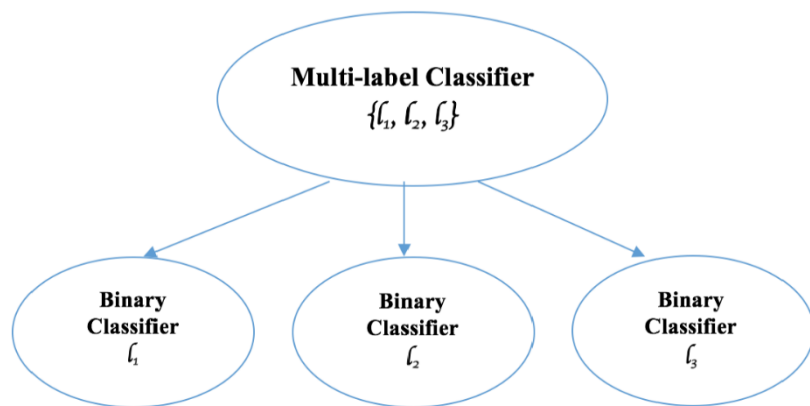
- 손으로 쓴 숫자 인식 문제

(MNIST Data, Modified National Institute of Standards and Technology)



다중 레이블 분류 Multi-Label Classification

- 관측값을 여러 개 클래스에 할당하는 문제
 - 바다와 석양을 찍은 사진 → "바다", "석양"
 - 어드벤처 영화 → "판타지", "드라마"
- n 개 클래스 레이블 분류 문제를 n 개의 이진 클래스 분류 문제로 바꿔서 해결 할수도 있음.



텍스트 분류 어플리케이션

- 뉴스의 성향을 판별 Political Orientation Identification
- 감정 분석 Sentiment Analysis
- 뉴스 카테고리 분석
- NER Named-Entity Recognition

< NER 예시 >

The California[Location]-based company, owned and operated by technology entrepreneur Elon Musk[Person], has proposed an orbiting digital communications array that would eventually consist of 4,425[Quantity] satellites, the documents filed on Tuesday[Date] show.

Bayes Theorem

조건부 확률 Conditional Probability

■ 사건 Event

- 사건 예1) 내일 비가 올 경우
- 사건 예2) 포커 카드팩에서 2장 뽑았는데 둘다 킹인 경우
- 사건 예3) 어떤 사람이 암에 걸린 경우

■ 조건부 확률

: 어떤 사건 B가 발생했다는 것을 알았을 때,
A가 일어날(일어났을) 확률

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

베이지 정리 Bayes' Theorem

- 베이지 정리는 조건부 확률 $P(A | B)$ 를 다음과 같이 계산

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

$P(B | A)$ 는 사건 A가 주어졌을 때 사건 B가 발생할 확률
 $P(B)$ 와 $P(A)$ 는 각각 사건 A와 B가 발생할 확률

베이지 정리 Bayes' Theorem

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

- ① $P(A)$ 사전(Prior) 확률
- ② $P(A|B)$ 사후(Posterior) 확률
- ③ $P(B)$ 증거(Evidence)
- ④ $P(B|A)$ 유사가능도(Likelihood)

베이지 정리 예시 : 동전

- 두 개의 동전 U, F 가 있다고 가정하자.
- 동전 U 는 던져서 앞면이 나올 확률이 90%이고, 동전 F 는 앞면이 나올 확률이 50%이다.
- 두 동전 중 하나를 임의로 선택해서 던졌을 때 결과가 앞면이 나왔다면, 이 동전이 U 일 확률은?

$$P(U|H) = ?$$

베이즈 정리 예시 : 동전

- 두 개의 동전 U, F 가 있다고 가정하자.
- 동전 U 는 던져서 앞면이 나올 확률이 90%이고, 동전 F 는 앞면이 나올 확률이 50%이다.
- 두 동전 중 하나를 임의로 선택해서 던졌을 때 결과가 앞면이 나왔다면, 이 동전이 U 일 확률은?

$$P(U|H) = \frac{P(H|U)P(U)}{P(H)}$$

$P(H|U)$ 는 90%, $P(U)$ 는 50%, 하지만 $P(H) = ?$

베이즈 정리 예시 : 동전

- $P(H)$, 즉 앞면이 나올 확률은
두 동전 모두를 고려해야 얻을 수 있다.
 - 동전 U 를 던져서 앞면을 얻을 확률
 - 동전 F 를 던져서 앞면을 얻을 확률

$$\begin{aligned} P(U|H) &= \frac{P(H|U)P(U)}{P(H)} = \frac{P(H|U)P(U)}{P(H|U)P(U) + P(H|F)P(F)} \\ &= \frac{0.9 * 0.5}{0.9 * 0.5 + 0.5 * 0.5} \end{aligned}$$

베이지 정리 예시 : 암 진단

- 내과 의사가 10,000명의 사람을 대상으로 다음과 같은 암 진단 테스트 결과를 작성했다.

	Cancer	No Cancer	Total
Test Positive	80	900	980
Test Negative	20	9000	9020
Total	100	9900	10000

- 100명의 암 환자 중 20명은 진단이 잘못 됐고, 건강한 사람 9,900명 중 900명의 진단을 잘못 됐다.
- 새로운 사람 1명의 진단 결과가 양성이라면, 이 사람이 실제 암에 걸렸을 확률은?

$$P(C|Pos) = ?$$

베이지 정리 예시 : 암 진단

	Cancer	No Cancer	Total
Test Positive	80	900	980
Test Negative	20	9000	9020
Total	100	9900	10000

$$P(C|Pos) = \frac{P(Pos|C)P(C)}{P(Pos)} = \frac{\frac{80}{100} * \frac{100}{10000}}{\frac{980}{10000}} = 8.16\%$$

베이지 정리 예시 : 공장

- 어느 공장의 A, B, C 세 대의 기계가 전체 전구 생산량의 35%, 20%, 45%씩을 만들어 내고 있다.
- 각 기계에서 생산된 전구의 불량률은 A가 1.5%, B가 1%, C가 2%다.
- 이 공장에서 생산된 전구에서 불량품이 발견됐다고 하자 (이 사건을 D로 표현)
- 각각의 기계 A, B, C가 이 전구를 생산했을 확률을 계산하면 얼마씩일까?

베이지 정리 예시 : 공장

$$P(A|D) = \frac{P(D|A)P(A)}{P(D)} = \frac{P(D|A)P(A)}{P(D|A)P(A) + P(D|B)P(B) + P(D|C)P(C)} = 32.3\%$$

$$P(B|D) = \frac{P(D|B)P(B)}{P(D)} = \frac{P(D|B)P(B)}{P(D|A)P(A) + P(D|B)P(B) + P(D|C)P(C)} = 12.3\%$$

$$P(C|D) = \frac{P(D|C)P(C)}{P(D)} = \frac{P(D|C)P(C)}{P(D|A)P(A) + P(D|B)P(B) + P(D|C)P(C)} = 55.4\%$$

불량품이 발생했을 때 가장 의심해볼 만 한 곳은 C

(단순히 가장 확률이 높은 곳을 찾고자 할 때는 $P(D)$ 는 계산 생략 가능!)

Naïve Bayes

나이브 베이즈 Naïve Bayes

- 데이터가 각 클래스에 속할 확률값을 계산해
모든 클래스를 대상으로 확률분포를 예측하는
확률 기반 분류기

나이브 베이즈의 동작 원리

- n 개의 특성을 지닌 샘플 데이터 $\mathbf{x}$ 가 주어졌을 때
(특성 벡터 $\mathbf{x} = (x_1, x_2, \dots, x_n)$)

나이브 베이즈의 목표는 이 샘플 데이터가 K 개의 클래스 $y_1, y_2, \dots, y_K$ 중 하나에 속할 확률을 결정

- $P(y_k | \mathbf{x})$ 또는 $P(y_k | x_1, x_2, \dots, x_n)$

$$\operatorname{argmax}_{y_k} P(y_k | \mathbf{x})$$

나이브 베이즈의 동작 원리

- n 개의 특성을 지닌 샘플 데이터 $\mathbf{x}$ 가 주어졌을 때
(특성 벡터 $\mathbf{x} = (x_1, x_2, \dots, x_n)$)

나이브 베이즈의 목표는 이 샘플 데이터가 K 개의 클래스 $y_1, y_2, \dots, y_K$ 중 하나에 속할 확률을 결정

- $P(y_k | \mathbf{x})$ 또는 $P(y_k | x_1, x_2, \dots, x_n)$

$$\operatorname{argmax}_{y_k} P(y_k | \mathbf{x}) \quad \rightarrow \quad \operatorname{argmax}_{y_k} \frac{P(\mathbf{x} | y_k) P(y_k)}{P(\mathbf{x})}$$

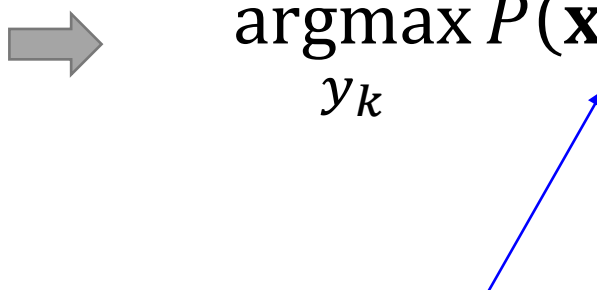
$$P(\mathbf{x} | y_k) = P(x_1 | y_k) * P(x_2 | y_k) * \dots * P(x_n | y_k)$$

나이브 베이즈의 동작 원리

$P(\mathbf{x})$ 는 모든 클래스에 대해 분모에 공통으로 존재하여
클래스를 결정 하는데 영향이 없으므로 생략 가능

$$\operatorname{argmax}_{y_k} P(y_k | \mathbf{x}) \quad \Rightarrow \quad \operatorname{argmax}_{y_k} P(\mathbf{x} | y_k) P(y_k)$$

$P(\mathbf{x} | y_k) = P(x_1 | y_k) * P(x_2 | y_k) * \cdots * P(x_n | y_k)$



Naïve Bayes Example

나이브 베이즈 동작 예시

- 4개의 이메일로 나이브 베이즈 모델을 학습 시켜, 새로운 이메일이 스팸일지 아닐지 예측해 보자.

	ID	Terms in e-mail	Is spam
Training data	1	Click win prize	Yes
	2	Click meeting setup meeting	No
	3	Prize free prize	Yes
	4	Click prize free	Yes
Testing case	5	Free setup meeting free	?

나이프 베이즈 동작 예시

	ID	Terms in e-mail	Is spam
Training data	1	Click win prize	Yes
	2	Click meeting setup meeting	No
	3	Prize free prize	Yes
	4	Click prize free	Yes
Testing case	5	Free setup meeting free	?

스팸 메일을 S 정상 메일을 NS로 정의하고
특성들로 단어 출현 정보를 사용하면
각 클래스에 속할 확률은?

$$P(S | free, setup, meeting, free)$$

$$P(NS | free, setup, meeting, free)$$

나이브 베이즈 동작 예시

$$\operatorname{argmax}_{y \in \{S, NS\}} P(y | \text{free}, \text{setup}, \text{meeting}, \text{free})$$



베이즈 정리

$$\operatorname{argmax}_{y \in \{S, NS\}} \frac{P(\text{free}, \text{setup}, \text{meeting}, \text{free} | y) P(y)}{P(\text{free}, \text{setup}, \text{meeting}, \text{free})}$$

독립 가정
(나이브)

$$\text{where } P(\text{free}, \text{setup}, \text{meeting}, \text{free} | y) = P(\text{free} | y) * P(\text{setup} | y) * P(\text{meeting} | y) * P(\text{free} | y)$$



$P(\text{free}, \text{setup}, \text{meeting}, \text{free})$ 는 모든 클래스 공통

$$\operatorname{argmax}_{y \in \{S, NS\}} P(\text{free} | y) * P(\text{setup} | y) * P(\text{meeting} | y) * P(\text{free} | y) P(y)$$

Likelihood 값 구하기

$$\operatorname{argmax}_{y \in \{S, NS\}} P(\text{free}|y) * P(\text{setup}|y) * P(\text{meeting}|y) * P(\text{free}|y) P(y)$$

“Likelihood 기본식”

$$P(\text{free}|y = S) = \frac{S \text{ 클래스에서 free라는 단어가 출현한 횟수}}{S \text{ 클래스 내 단어 출현 총 횟수}} = \frac{2}{9}$$

만약 $P(x_i | y_k)$ 가 0이라면 어떻게 될까?

즉, 단어 하나가 해당 클래스에서 관측되지 않은 경우!

이렇게 되면 다른 단어의 Likelihood 값에 상관없이 전체가 0이 된다.

값이 0이 되지 않도록 하는 방법 “스무딩”을 적용

Likelihood 값 구하기 (스무딩)

라플라스 스무딩 (Laplace Smoothing)

: 모든 클래스에서 단어들이 한번씩 출현했다고 가정

$$P(\text{free} | y = S) = \frac{2 + \textcircled{1}}{9 + \textcircled{6}} = \frac{3}{15}$$

단어들의 종류가 총 6개

(click, win, prize, meeting, setup, free)

이들 단어가 S 클래스에서 한번씩은 등장했다는 가정에 따라
단어당 한번씩 해서 6을 더해준다.

→ 그냥 분모에도 1을 더해주거나

혹은 분모는 어차피 모든 단어에 대해 같으니 아무것도 더해주지 않는 방법도 가능하다.

가정에 따라
free 한번 출현 횟수를
더해준다.

나이브 베이즈 동작 예시

이제 본격적으로 주어진 메일이 스팸인지 아닐지 판별해 봅시다.

	ID	Terms in e-mail	Is spam
Training data	1	Click win prize	Yes
	2	Click meeting setup meeting	No
	3	Prize free prize	Yes
	4	Click prize free	Yes
Testing case	5	Free setup meeting free	?

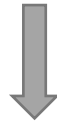
판별식 : $\operatorname{argmax}_{y \in \{S, NS\}} P(\text{free}|y) * P(\text{setup}|y) * P(\text{meeting}|y) * P(\text{free}|y) P(y)$

나이브 베이즈 동작 예시

1. 스팸일 경우

	ID	Terms in e-mail	Is spam
Training data	1	Click win prize	Yes
	2	Click meeting setup meeting	No
	3	Prize free prize	Yes
	4	Click prize free	Yes
Testing case	5	Free setup meeting free	?

$$P(\text{free}|S) * P(\text{setup}|S) * P(\text{meeting}|S) * P(\text{free}|S) P(S)$$



$$P(\text{free}|S) = \frac{2+1}{9+6} = \frac{3}{15}$$

$$P(S) = \frac{3}{4}$$

$$P(\text{setup}|S) = \frac{0+1}{9+6} = \frac{1}{15}$$

$$\frac{3}{15} * \frac{1}{15} * \frac{1}{15} * \frac{3}{15} * \frac{3}{4} = \frac{1}{7500}$$

$$P(\text{meeting}|S) = \frac{0+1}{9+6} = \frac{1}{15}$$

$$P(\mathbf{x} | S)P(S) \approx 0.000133$$

나이브 베이즈 동작 예시

2. 스팸이 아닐 경우

	ID	Terms in e-mail	Is spam
Training data	1	Click win prize	Yes
	2	Click meeting setup meeting	No
	3	Prize free prize	Yes
	4	Click prize free	Yes
Testing case	5	Free setup meeting free	?

$$P(\text{free}|\text{NS}) * P(\text{setup}|\text{NS}) * P(\text{meeting}|\text{NS}) * P(\text{free}|\text{NS}) P(\text{NS})$$



$$P(\text{free}|\text{NS}) = \frac{0+1}{4+6} = \frac{1}{10}$$

$$P(\text{NS}) = \frac{1}{4} \quad P(\text{setup}|\text{NS}) = \frac{1+1}{4+6} = \frac{2}{10}$$

$$P(\text{meeting}|\text{NS}) = \frac{2+1}{4+6} = \frac{3}{10}$$

$$\frac{1}{10} * \frac{2}{10} * \frac{3}{10} * \frac{1}{10} * \frac{1}{4} = \frac{3}{20000}$$

$$P(\mathbf{x}|\text{NS})P(\text{NS}) \approx 0.00015$$

나이브 베이즈 동작 예시

1. 스팸일 경우

$$P(\mathbf{x} | S)P(S) \approx 0.000133$$

2. 스팸이 아닐 경우

$$P(\mathbf{x} | NS)P(NS) \approx 0.00015$$

스팸이 아닐 경우의 값이 더 크기 때문에
주어진 메일은 스팸이 아닐 확률이 더 크다고 결론!

나이브 베이즈 동작 예시

나아가, 실제 주어진 메일이 스팸일 확률과 그렇지 않을 확률도 계산 가능

$$P(\mathbf{x} | S)P(S) \approx 0.00013$$

$$P(\mathbf{x} | NS)P(NS) \approx 0.00015$$

$$P(S | \mathbf{x}) = \frac{P(\mathbf{x} | S)P(S)}{P(\mathbf{x})} = \frac{P(\mathbf{x} | S)P(S)}{P(\mathbf{x} | S)P(S) + P(\mathbf{x} | NS)P(NS)} = \frac{0.00013}{0.00013 + 0.00015} = 46.42\%$$

$$P(NS | \mathbf{x}) = 1 - P(S | \mathbf{x}) = 53.57\%$$

주어진 이메일은 스팸이 아닐 확률이 더 높다!

Log() 사용

$$P(\mathbf{x}|y_k) = P(x_1|y_k) * P(x_2|y_k) * \dots * P(x_n|y_k)$$

단어들이 엄청 많다고 생각해보자.

Likelihood가 0~1사이 값을 가지고

이들을 곱하고 곱하고 곱하다 보면 매우 작은 값을 가지게 될 것이다.

컴퓨터가 처리할 수 있는 작은 소수자리가 제한되어 있어

일정 수준이 넘게 작아지면 그냥 다 똑같은 값으로 치환된다. (Overflow 문제)

따라서 Log를 취해서 곱셈이 아닌 덧셈으로 바꿔서 이를 방지한다.

$$P(\mathbf{x}|y_k) = \exp(\text{Log}(P(x_1|y_k)) + \text{Log}(P(x_2|y_k)) \dots \text{Log}(P(x_n|y_k)))$$

Naïve Bayes in Python

Naïve Bayes in Python

이메일 데이터를 분석해서
스팸인지 일반 메일인지를 구분하는
Naïve Bayes를 구현해 보자!

데이터 다운로드

- 스팸/메일 데이터

- http://nlp.cs.aueb.gr/software_and_datasets/Enron-Spam/preprocessed/enron1.tar.gz

데이터 읽어서 확인해 보기

```
file_path = 'enron1/ham/0007.1999-12-14.farmer.ham.txt'

with open(file_path, 'r') as infile:

    ham_sample = infile.read()

print(ham_sample)
```

Subject: mcmullen gas for 11 / 99
jackie ,
since the inlet to 3 river plant is shut in on 10 / 19 / 99 (the last day of
flow) :
at what meter is the mcmullen gas being diverted to ?
at what meter is hpl buying the residue gas ? (this is the gas from teco ,
vastar , vintage , tejones , and swift)
i still see active deals at meter 3405 in path manager for teco , vastar ,
vintage , tejones , and swift
i also see gas scheduled in pops at meter 3404 and 3405 .
please advice . we need to resolve this as soon as possible so settlement
can send out payments .
thanks

1. 입력 데이터 만들기

- 현재는 메일과 스팸이 폴더로 각각 구분되어 텍스트파일(txt)에 저장되어 있다.
 - 일반 메일 폴더: ham
 - 스팸 메일 폴더: spam
- 데이터를 불러와 Python 변수에 저장하도록 하자.

1. 입력 데이터 만들기

- 스팸 데이터 불러오기

```
import glob
import os

mails = []
labels = []

file_path = 'enron1/spam/'

for filename in glob.glob(os.path.join(file_path, '*.txt')):
    with open(filename, 'r', encoding = "ISO-8859-1") as infile:
        mails.append(infile.read())
        labels.append(1)
```


1. 입력 데이터 만들기

- 일반 데이터 불러오기

```
file_path = 'enron1/ham/'  
  
for filename in glob.glob(os.path.join(file_path, '*.txt')):  
    with open(filename, 'r', encoding = "ISO-8859-1") as infile:  
        mails.append(infile.read())  
        labels.append(0)
```

2. 데이터 전처리

- 숫자와 구두점 표기 제거
- 사람 이름 제거
- 불용어(Stop word) 제거
- 표제어 원형 복원

<앞서 사용했던 코드를 다시 사용하자>

3. 학습/테스트 데이터 나누기

- 파이썬의 `train_test_split` 함수를 쓰면 쉽게 나눌 수 있다.
 - 원하는 비율로 데이터들을 임의로 두 그룹에 할당한다.

```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(cleaned_mails,
                                                    labels,
                                                    test_size=0.33,
                                                    random_state=0)
```

※ 주요 파라미터

- `test_size` : 전체 데이터에서 테스트 데이터 비중 결정
- `random_state` : 임의 선택 결과를 주어진 숫자에 따라 결정할 수 있음
(여러 번 실험하는 상황에서 같은 데이터에 대해 비교하고 싶을 때 유용)

4. 벡터화

벡터화 시 Stopword 여부와 max_features는 얼마든지 튜닝 가능

```
from sklearn.feature_extraction.text import CountVectorizer  
  
cv_train = CountVectorizer(stop_words="english", max_features=500)  
  
term_docs_train = cv_train.fit_transform(X_train)  
  
term_docs_test = cv_train.transform(X_test)
```

Test 데이터에 대해서는 transform을 사용함에 주의

X_train[0]

're may nomination george the following are
on no the nomination is at the pay meter in
a not been listed on the spreadsheet it ha
sk group a of comstock s nomination is and
hase is not listed on the spreadsheet it is
a file by daren to the best of my knowledge
rchase bob enron north america corp from ge
t cotton hou ect ect cc vance l taylor hou

print(term_docs_train[0])

(0, 240)	1
(0, 146)	1
(0, 283)	1
(0, 441)	1
(0, 96)	2
(0, 298)	1
(0, 246)	1
(0, 122)	1
(0, 245)	1
(0, 87)	1
(0, 475)	5
(0, 297)	2
(0, 38)	2
(0, 369)	2

문서번호

문서 내
빈도수

단어번호

fit_transform()

4. 벡터화

단어 번호에 해당하는 단어를 보고 싶다면

`get_feature_names()` 함수를 사용!

```
print(term_docs_train[0])
```

(0, 240)	1
(0, 146)	1
(0, 283)	1
(0, 441)	1
(0, 96)	2
(0, 298)	1
(0, 246)	1
(0, 122)	1
(0, 245)	1
(0, 87)	1
(0, 475)	5
(0, 297)	2
(0, 38)	2
(0, 369)	2



```
feature_names = cv_train.get_feature_names()  
print(feature_names[240])  
print(feature_names[357])  
print(feature_names[125])
```

```
ll  
real  
entered
```

5. 학습 데이터로 모델 학습

```
from sklearn.naive_bayes import MultinomialNB  
  
clf = MultinomialNB(alpha=1.0, fit_prior=True)  
  
clf.fit(term_docs_train, Y_train)
```

■ MultinomialNB : Naïve Bayes 분류기 클래스 생성자

- alpha : 스무딩 factor
- fit_prior : True면 학습 데이터 내 분포로 Prior 설정

■ __.fit () : 학습 모델 생성

- 첫번째에 Feature 벡터가 들어가고, 두번째에 Label 리스트 입력

6. 테스트 데이터로 예측 결과 생성

- 클래스 별 예측된 확률 값 계산

```
prediction_prob = clf.predict_proba(term_docs_test)
prediction_prob
```

```
array([[1.00000000e+00, 2.12716600e-10],
       [1.00000000e+00, 2.72887131e-75],
       [6.34671963e-01, 3.65328037e-01],
       ...,
       [1.00000000e+00, 2.66925045e-14],
       [2.06007664e-42, 1.00000000e+00],
       [9.99999973e-01, 2.72553601e-08]])
```

클래스 0 (정상)에 대한 확률

클래스 1 (스팸)에 대한 확률

6. 테스트 데이터로 예측 결과 생성

- 계산된 확률 값에 기반해 최종 예측 결과값 생성
 - 0.5를 임계치(Threshold)로 사용
(예: prediction value가 0.6 → 스팸
prediction value가 0.2 → 정상)

```
prediction = clf.predict(term_docs_test)
prediction
```

```
array([0, 0, 0, ..., 0, 1, 0])
```


Lab 2-2: Naïve Bayes Classifier

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

Performance Evaluation

Recall!

이메일 데이터를 분석해서
스팸인지 일반 메일인지를 구분하는
Naïve Bayes를 구현해 보자!

데이터 다운로드

- 스팸/메일 데이터
 - http://nlp.cs.aueb.gr/software_and_datasets/Enron-Spam/preprocessed/enron1.tar.gz
- 다운로드, 압축풀기, 확인해보기 등

1. 입력 데이터 만들기

- 현재는 메일과 스팸이 폴더로 각각 구분되어 텍스트파일(txt)에 저장되어 있다.
 - 일반 메일 폴더: ham
 - 스팸 메일 폴더: spam
- 데이터를 불러와 Python 변수에 저장하도록 하자.

1. 입력 데이터 만들기

- 스팸 데이터 불러오기

```
import glob
import os

mails = []
labels = []

file_path = 'enron1/spam/'

for filename in glob.glob(os.path.join(file_path, '*.txt')):
    with open(filename, 'r', encoding = "ISO-8859-1") as infile:
        mails.append(infile.read())
        labels.append(1)
```

1. 입력 데이터 만들기

- 일반 데이터 불러오기

```
file_path = 'enron1/ham/'  
  
for filename in glob.glob(os.path.join(file_path, '*.txt')):  
    with open(filename, 'r', encoding = "ISO-8859-1") as infile:  
        mails.append(infile.read())  
        labels.append(0)
```


2. 데이터 전처리

- 숫자와 구두점 표기 제거
- 사람 이름 제거
- 불용어(Stop word) 제거
- 표제어 원형 복원

<앞서 사용했던 코드를 다시 사용하자>

3. 학습/테스트 데이터 나누기

- 파이썬의 `train_test_split` 함수를 쓰면 쉽게 나눌 수 있다.
 - 원하는 비율로 데이터들을 임의로 두 그룹에 할당한다.

```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(cleaned_mails,
                                                    labels,
                                                    test_size=0.33,
                                                    random_state=0)
```

※ 주요 파라미터

- `test_size` : 전체 데이터에서 테스트 데이터 비중 결정
- `random_state` : 임의 선택 결과를 주어진 숫자에 따라 결정할 수 있음
(여러 번 실험하는 상황에서 같은 데이터에 대해 비교하고 싶을 때 유용)

4. 벡터화

벡터화 시 Stopword 여부와 max_features는 얼마든지 튜닝 가능

```
from sklearn.feature_extraction.text import CountVectorizer  
  
cv_train = CountVectorizer(stop_words="english", max_features=500)  
  
term_docs_train = cv_train.fit_transform(X_train)  
  
term_docs_test = cv_train.transform(X_test)
```

Test 데이터에 대해서는
transform을 사용함에 주의

X_train[0]

're may nomination george the following are
on no the nomination is at the pay meter in
a not been listed on the spreadsheet it ha
sk group a of comstock s nomination is and
hase is not listed on the spreadsheet it is
a file by daren to the best of my knowledge
rchase bob enron north america corp from ge
t cotton hou ect ect cc vance l taylor hou

print(term_docs_train[0])

(0, 240)	1
(0, 146)	1
(0, 283)	1
(0, 441)	1
(0, 96)	2
(0, 298)	1
(0, 246)	1
(0, 122)	1
(0, 245)	1
(0, 87)	1
(0, 475)	5
(0, 297)	2
(0, 38)	2
(0, 369)	2

문서번호

문서 내
빈도수

단어번호

fit_transform()

4. 벡터화

단어 번호에 해당하는 단어를 보고 싶다면

`get_feature_names()` 함수를 사용!

```
print(term_docs_train[0])
```

(0, 240)	1
(0, 146)	1
(0, 283)	1
(0, 441)	1
(0, 96)	2
(0, 298)	1
(0, 246)	1
(0, 122)	1
(0, 245)	1
(0, 87)	1
(0, 475)	5
(0, 297)	2
(0, 38)	2
(0, 369)	2



```
feature_names = cv_train.get_feature_names()  
print(feature_names[240])  
print(feature_names[357])  
print(feature_names[125])
```

```
ll  
real  
entered
```

5. 학습 데이터로 모델 학습

```
from sklearn.naive_bayes import MultinomialNB  
  
clf = MultinomialNB(alpha=1.0, fit_prior=True)  
  
clf.fit(term_docs_train, Y_train)
```

■ MultinomialNB : Naïve Bayes 분류기 클래스 생성자

- alpha : 스무딩 factor
- fit_prior : True면 학습 데이터 내 분포로 Prior 설정

■ __.fit () : 학습 모델 생성

- 첫번째에 Feature 벡터가 들어가고, 두번째에 Label 리스트 입력

6. 테스트 데이터로 예측 결과 생성

- 클래스 별 예측된 확률 값 계산

```
prediction_prob = clf.predict_proba(term_docs_test)
prediction_prob
```

```
array([[1.00000000e+00, 2.12716600e-10],
       [1.00000000e+00, 2.72887131e-75],
       [6.34671963e-01, 3.65328037e-01],
       ...,
       [1.00000000e+00, 2.66925045e-14],
       [2.06007664e-42, 1.00000000e+00],
       [9.99999973e-01, 2.72553601e-08]])
```

클래스 0 (정상)에 대한 확률

클래스 1 (스팸)에 대한 확률

6. 테스트 데이터로 예측 결과 생성

- 계산된 확률 값에 기반해 최종 예측 결과값 생성
 - 0.5를 임계치(Threshold)로 사용
(예: prediction value가 0.6 → 스팸
prediction value가 0.2 → 정상)

```
prediction = clf.predict(term_docs_test)
prediction
```

```
array([0, 0, 0, ..., 0, 1, 0])
```

7. 성능 평가 : 성능 척도

- 테스트 데이터에 대해서 모델이 예측한 분류 결과는 Confusion Matrix라고 불리는 아래 테이블과 같이 정리될 수 있다.

		Predicted	
		Negative	Positive
Actual	Negative	TN	FP
	Positive	FN	TP

TN = True Negative
FP = False Positive
FN = False Negative
TP = True Positive

- 예측결과가 맞으면 T, 틀리면 F로 시작
- 양성으로 예측한 경우 P, 음성으로 예측한 경우 N으로 마침

7. 성능 평가 : Confusion Matrix

- 예) 환자 500명이 있을 때,
정상 vs. 암을 분류하는 기계학습 모델이 있었다면
다음과 같이 결과를 정리해 볼 수 있을 것이다.

		분류 결과	
		정상 (음성)	암 (양성)
실제	정상 (음성)	450	30
	암 (양성)	10	10

7. 성능 평가 : Accuracy (정확도)

- 가장 일반적으로 많이 사용하는 성능 지표
- 전체 케이스 중에 올바르게 예측한 케이스의 비율

- $$\text{Accuracy} = \frac{TN+TP}{TN+TP+FP+FN}$$

		분류 결과	
		정상 (음성)	암 (양성)
실제	정상 (음성)	450	30
	암 (양성)	10	10

<정확도>
 $460/500 = 92\%$

7. 성능 평가 : Accuracy (정확도)

		분류 결과	
		정상 (음성)	암 (양성)
실제	정상 (음성)	450	30
	암 (양성)	10	10

<정확도>

$$460/500 = 92\%$$

- 정확도는 각 클래스 별 자세한 성능 결과가 공개져 있다.
 - 예) 암 환자를 정말 암이라고 했을 확률?
 - 예) 정상 환자를 정말 정상이라고 했을 확률?
 - 예) 암이라고 분류했을 때 진짜 암 환자가 맞을 확률?

클래스 별 성능을 평가해 볼 수 있는 성능 척도들이 존재한다.

7. 성능 평가 : 클래스 별 성능 척도

- 클래스 별 성능 척도 (이진 분류 문제)
 - Precision
 - Recall (= Sensitivity, True Positive Rate)
 - True Negative Rate (= Specificity)

다중 클래스 문제의 경우 관심 클래스를 Positive로 두고
그 외 클래스들을 Negative로 두어서 이진화 한 후
클래스 별 성능 척도를 적용할 수 있다.

7. 성능 평가 : Precision (정밀도)

- 스팸이라고 예측한 메일들 중 진짜 스팸 메일의 비율
- 검색 엔진과 같은 Application에서 중요
- $$\text{Precision} = \frac{TP}{TP+FP}$$

		Predicted	
		Negative	Positive
Actual	Negative	TN	FP
	Positive	FN	TP

TN = True Negative
FP = False Positive
FN = False Negative
TP = True Positive

7. 성능 평가 : Recall (재현율)

- 관심 클래스(Positive)에 속하는 데이터를 얼마나 잘 예측해 골라 냈는지?
 - 예) 진짜 스팸 메일 중에서 스팸이라고 잘 예측해낸 비율
- Sensitivity, True Positive Rate(TPR)이라고도 한다.
- $\text{Recall}(= \text{Sensitivity}) = \frac{TP}{TP+FN}$

		Predicted	
		Negative	Positive
Actual	Negative	TN	FP
	Positive	FN	TP

TN = True Negative
FP = False Positive
FN = False Negative
TP = True Positive

7. 성능 평가 : TNR

- 음성 클래스(Negative)에 속하는 데이터를 얼마나 잘 예측해 골라 냈는지?
 - 예) 진짜 정상 메일 중에서 정상이라고 잘 예측해낸 비율
- Specificity** 라고도 한다. (Negative에 대한 재현율)
- $$\text{Specificity} = \frac{TN}{TN+FP}$$

		Predicted	
		Negative	Positive
Actual	Negative	TN	FP
	Positive	FN	TP

TN = True Negative
FP = False Positive
FN = False Negative
TP = True Positive

7. 성능 평가 : 척도 간의 Trade-Off

- 이상적으로는 모든 척도에서 수치가 다 높으면 좋은 모델
- 하지만 척도 간의 Trade-Off 관계가 있음을 고려해야 한다.
 - Precision vs. Recall
 - 정말 Positive라고 확신할 수 있는 경우만 Positive로 예측하게 되면 전체 Positive 데이터 중 많은 경우를 놓칠 수 있다.
 - Sensitivity vs. Specificity
 - Positive 예측을 더 많이 하면 Positive에 대한 재현율(Sensitivity)은 좋아질 수 있으나, 상대적으로 Negative에 대한 재현율(Specificity)는 나빠질 수 있다.

7. 성능 평가 : 종합 성능 척도

- 정확도와 비슷하게 **모델의 전반적인 성능**을 요약해 주는 것을 클래스 별 성능 척도들을 토대로 계산할 수 있다.
 - F1 Score
 - AUC (Area Under the Curve)
 - ROC Curve

7. 성능 평가 : F1-score

■ 정밀도와 재현율의 조화 평균 (Harmonic mean)

- $F1 = 2 * \frac{precision * recall}{precision + recall}$

- $Recall = \frac{TP}{TP + FN}$

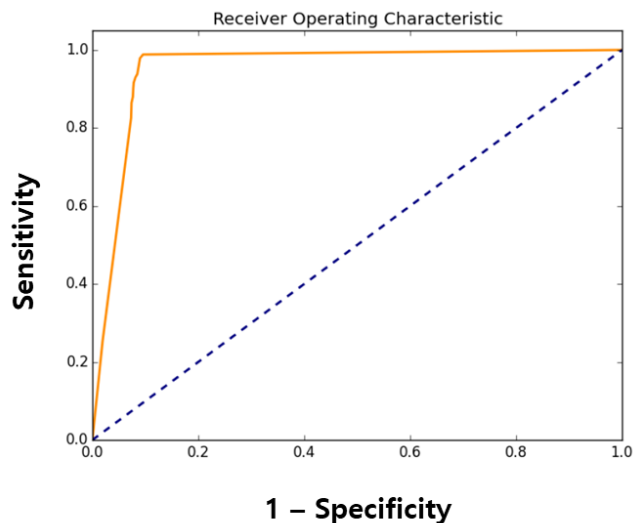
- $Precision = \frac{TP}{TP + FP}$

		Predicted	
		Negative	Positive
Actual	Negative	TN	FP
	Positive	FN	TP

TN = True Negative
FP = False Positive
FN = False Negative
TP = True Positive

7. 성능 평가 : AUC

- AUC를 구하기 위해서는 ROC Curve라고 하는 그래프를 먼저 그린다.
 - X축 : $1 - \text{Specificity}$ (=False Positive Rate)
 - Y축 : Sensitivity (= TPR)
 - 임계치를 조절해 가면서 (예: 0, 0.1, 0.2 ... 1), 각 임계치 별 Sensitivity와 Specificity 값을 좌표로 찍으면 다음과 같은 그래프를 얻을 수 있다.



Specificity가 감소할 수록
(X축의 1- Specificity가 증가할 수록)

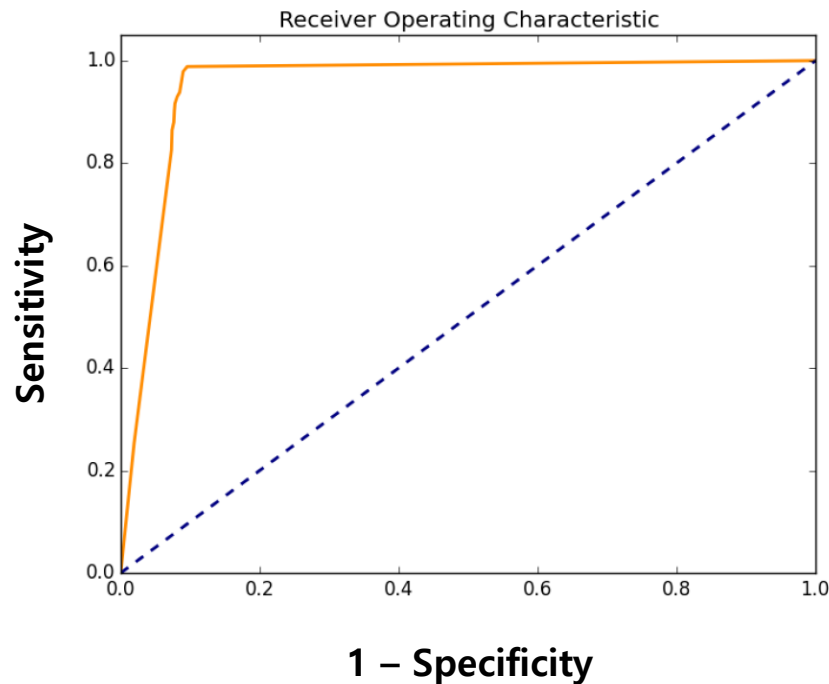
Sensitivity가 증가하는데

Sensitivity 최댓값(1)에 빨리 도달할 수록
좋은 모델이라 할 수 있다. (Specificity 손해가 최소)

→ 빨리 도달 할수록 ROC Curve 아래 면적이 넓어진다.

7. 성능 평가 : AUC

- AUC (Area Under the Curve)는 ROC Curve 아래 면적을 의미하며 값이 크면 클 수록 좋은 값이다.
 - 최소 : 0, 최대 : 1



7. 성능 평가 in Python

■ Confusion Matrix

```
from sklearn.metrics import confusion_matrix  
  
prediction = clf.predict(term_docs_test)  
confusion_matrix(Y_test, prediction, labels = [0, 1])
```

```
array([[1138,   85],  
       [  51,  433]], dtype=int64)
```

■ Accuracy

```
accuracy = clf.score(term_docs_test, Y_test)  
  
print('The accuracy using MultinomialNB is:{0:.1f}%'.format(accuracy*100))
```

```
The accuracy using MultinomialNB is:92.0%
```

7. 성능 평가 in Python

- Precision, Recall, F1 Score

```
from sklearn.metrics import precision_score, recall_score, f1_score  
  
print ( precision_score(Y_test, prediction, pos_label=1) )  
  
print ( recall_score(Y_test, prediction, pos_label=1) )  
  
print ( f1_score(Y_test, prediction, pos_label=1) )
```

0.8356890459363958

0.9166666666666666

0.8743068391866913

7. 성능 평가 in Python

■ ROC Curve

(1) 임계치에 따른 Sensitivity, Specificity 값 계산

```
import numpy as np
pos_prob = prediction_prob[:, 1]

thresholds = np.arange(0.0, 1.2, 0.1)

true_pos, true_neg = [0]*len(thresholds), [0]*len(thresholds)

for pred, y in zip(pos_prob, Y_test):
    for i, threshold in enumerate(thresholds):
        # if truth and prediction are both positive
        if pred >= threshold and y == 1:
            true_pos[i] += 1
        elif pred < threshold and y == 0:
            true_neg[i] += 1

sensitivity = [tp / np.sum(Y_test) for tp in true_pos]
cmlp_specificity = [1 - tn / (len(Y_test) - np.sum(Y_test))] for tn in true_neg]
```

7. 성능 평가 in Python

■ ROC Curve

(2) ROC Curve 그래프 그리기

```
import matplotlib.pyplot as plt
plt.figure()
lw = 2
plt.plot(cmpl_specificity, sensitivity, color='darkorange')

plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])

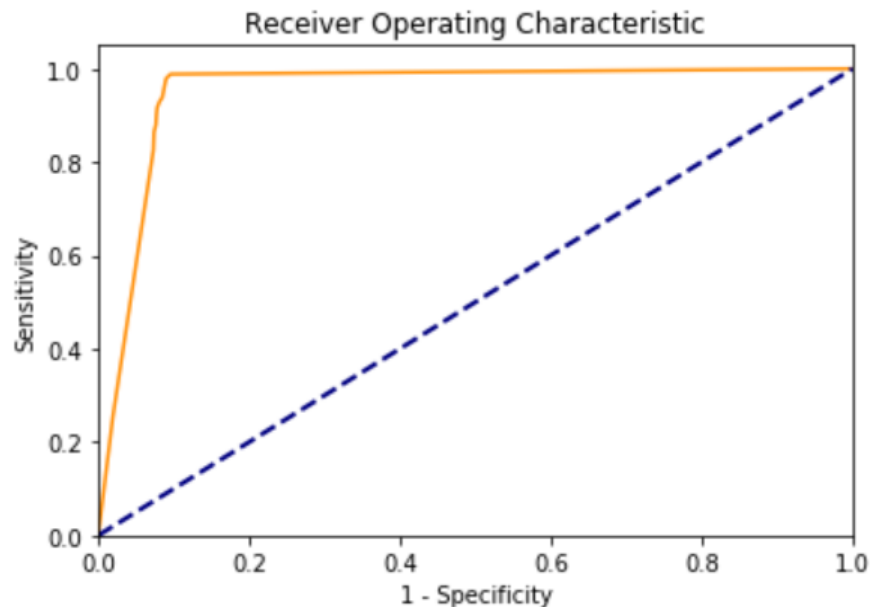
plt.xlabel('1 - Specificity')
plt.ylabel('Sensitivity')
plt.title('Receiver Operating Characteristic')
plt.show()
```


7. 성능 평가 in Python

■ AUC 계산

```
from sklearn.metrics import roc_auc_score  
roc_auc_score(Y_test, pos_prob)
```

0.9635659163552571



k-fold Cross Validation

이전 결과는 학습 데이터와 테스트 데이터를
각각 하나의 Set만 사용하여 얻은 결과 였다.



따라서 일반화 된 결과라 하기 어렵다.
(앞서 사용한 Test Set에서만 잘 작동할 수 있다.)



따라서 보다 일반적인 성능을 평가해 보기 위하여
K-Folds Cross Validation을 수행해 보자.

k-fold Cross Validation

```
from sklearn.model_selection import StratifiedKFold

k = 5
k_fold = StratifiedKFold(n_splits=k)

cleaned_mails_arr = np.array(cleaned_mails)
labels_arr = np.array(labels)

for trn_idx, test_idx in k_fold.split(cleaned_mails, labels):

    cv_train = CountVectorizer(stop_words="english", max_features=500)
    train_X = cleaned_mails_arr[trn_idx]
    train_Y = labels_arr[trn_idx]
    term_docs_train = cv_train.fit_transform(train_X)

    test_X = cleaned_mails_arr[test_idx]
    test_Y = labels_arr[test_idx]
    term_docs_test = cv_train.transform(test_X)

    clf = MultinomialNB(alpha=1.0)
    clf.fit(term_docs_train, train_Y)

    prediction = clf.predict(term_docs_test)
    print(classification_report(test_Y, prediction))
```

k-fold Cross Validation

<실행 결과>

	precision	recall	f1-score	support
0	0.95	0.93	0.94	735
1	0.83	0.88	0.86	300
avg / total	0.92	0.91	0.91	1035

	precision	recall	f1-score	support
0	0.96	0.96	0.96	735
1	0.89	0.89	0.89	300
avg / total	0.94	0.94	0.94	1035

	precision	recall	f1-score	support
0	0.94	0.96	0.95	734
1	0.89	0.86	0.87	300
avg / total	0.93	0.93	0.93	1034

	precision	recall	f1-score	support
0	0.96	0.92	0.94	734
1	0.82	0.91	0.86	300
avg / total	0.92	0.92	0.92	1034

	precision	recall	f1-score	support
0	0.97	0.87	0.92	734
1	0.75	0.93	0.83	300
avg / total	0.90	0.89	0.89	1034

K개 결과의 평균값을 구해서
모델 성능의 최종 결과로 삼는다.

이 최종 결과를 토대로
하이퍼 파라미터를 튜닝하거나
(예: Smoothing factor)
학습 모델을 선택한다.

TF-IDF

DTM

- Document-Term Matrix (DTM)

- Document들의 BoWs (Bag of words) 매트릭스
- BoW?
 - Bag of Words is a digitized representation of text data that focuses only on the frequency of the words appearing, without considering the order of the words at all

DTM with TF (Term Frequency)

- TF: Term Frequency

- 단어의 출현 빈도

- DTM with TF

	자동차	기차	커피	쿠키	...
문서 1	3	4	0	0	
문서 2	3	3	0	1	
문서 3	1	1	7	6	
...	...	...			

DTM with TF (Term Frequency)

■ 한계점

- Sparse representation

- Memory가 많이 필요함
- Stemming, lemmatization 등 필요

- 빈도 기반 접근법

- 중요하지 않은 단어들(예: the, a, an)의 출현 빈도가 높을 수 있음
- 예:
 - ✓ 문서 1: the best jam, the apple
 - ✓ 문서 2: the soccer player, the art
 - ✓ 문서 3: the delicious banana, the man
- "the"가 많이 있어서 문서 1, 2, 3은 유사하다고 알고리즘이 판단할 수 있음??
 - ✓ Stopword vs. important word

cat	the	quick	brown	fox	jumped	over	dog	bird	flew
0	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0

TF-IDF

- 대부분의 문서에서 일반적으로 많이 나오는 단어들은 TF 값이 높아도 문서의 고유한 특징을 잘 표현 하지 않을 수 있음!!
- 이점을 고려하여 여러 문서에서 나온 단어에 페널티를 주는 방식이 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF

- $tf(d, t)$: the number of occurrences of a specific word t in a particular document d
- $df(t)$: the number of documents in which the specific word t appears
- 예

Document1 : eat apple
Document2 : eat banana
Document3 : long and yellow banana
Document4 : I like banana

$tf(doc3, 'banana')=1$
 $df('banana') = 3$

Document1 : eat apple
Document2 : eat banana
Document3 : long and yellow banana banana
Document4 : I like banana

$tf(doc3, 'banana')=2$
 $df('banana') = 3$

TF-IDF

- $idf(d, t)$: inverse of $df(t)$ -> $idf(d, t) = \log \frac{n_D}{1 + df(t)}$

$$idf(d, t) = \log(n/df(t))$$

$n = 1,000,000$

단어 t	$df(t)$	$idf(d, t)$
word1	1	6
word2	100	4
word3	1,000	3
word4	10,000	2
word5	100,000	1
word6	1,000,000	0

VS.

$$idf(d, t) = n/df(t)$$

$n = 1,000,000$

단어 t	$df(t)$	$idf(d, t)$
word1	1	1,000,000
word2	100	10,000
word3	1,000	1,000
word4	10,000	100
word5	100,000	10
word6	1,000,000	1

TF-IDF

- TF-IDF (Term Frequency-Inverse Document Frequency)

$$tf-idf(d, t) = tf(d, t) * \log \frac{n_D}{1 + df(t)}$$

n_D : 전체 문서 수

$df(t)$: 단어 t 가 나온 문서 수

TF-IDF 예

TF

	자동차	기차	커피	쿠키	...
문서 1	3	4	0	0	
문서 2	3	3	0	1	
문서 3	1	1	7	6	

IDF

자동차	기차	커피	쿠키	...
$\log(3/4)$	$\log(3/4)$	$\log(3/2)$	$\log(3/3)$	

TF-IDF

	자동차	기차	커피	쿠키	...
문서 1	$3 \cdot \log(3/4)$	$4 \cdot \log(3/4)$	$0 \cdot \log(3/2)$	$0 \cdot \log(3/3)$	
문서 2	$3 \cdot \log(3/4)$	$3 \cdot \log(3/4)$	$0 \cdot \log(3/2)$	$1 \cdot \log(3/3)$	
문서 3	$1 \cdot \log(3/4)$	$1 \cdot \log(3/4)$	$7 \cdot \log(3/2)$	$6 \cdot \log(3/3)$	

Lab 2-3: Performance Evaluation

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.